

ig - SYNTAX ERROR

St x i v e n

a2z

DSA SHEET

- "495"

By - SYNTAX ERROR

Step 5: Strings

The PDF created by Abhishek Rathor (Instagram:SYNTAX ERROR)

PROBLEM 1

Problem Statement:

Given a valid **parentheses string**, remove the outermost parentheses from every primitive valid substring and return the result.

Java Code:

```
import java.util.Scanner;

public class RemoveOutermostParentheses {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a valid parentheses string: ");
        String s = scanner.next();
        scanner.close();

        System.out.println("Processed String: " +
            removeOuterParentheses(s));
    }

    public static String removeOuterParentheses(String s) {
        StringBuilder result = new StringBuilder();
        int count = 0;

        for (char c : s.toCharArray()) {
            if (c == '(' && count++ > 0) result.append(c);
            if (c == ')' && --count > 0) result.append(c);
        }
        return result.toString();
    }
}
```

Explanation:

1. **Iterate** over the string while maintaining a count of open parentheses.
2. **Skip** the first (and last) of each valid substring.
3. **Append the rest** to the result.

Example:

Input: " (()) (()) "

Output: " () () "

⌚ **Time Complexity: $O(N)$**
📦 **Space Complexity: $O(N)$**

PROBLEM 2

Problem Statement:

1. Reverse the order of words in a given string.
 2. Check if the given string is a palindrome.
-

Java Code:

```
import java.util.Scanner;

public class ReverseWordsPalindromeCheck {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a sentence: ");
        String s = scanner.nextLine();
        scanner.close();

        System.out.println("Reversed Words: " + reverseWords(s));
        System.out.println("Is Palindrome: " + isPalindrome(s));
    }

    public static String reverseWords(String s) {
        String[] words = s.trim().split("\\s+");
        StringBuilder reversed = new StringBuilder();
        for (int i = words.length - 1; i >= 0; i--) {
            reversed.append(words[i]).append(" ");
        }
        return reversed.toString().trim();
    }

    public static boolean isPalindrome(String s) {
        s = s.replaceAll("[^a-zA-Z0-9]", "").toLowerCase();
        int left = 0, right = s.length() - 1;
        while (left < right) {
            if (s.charAt(left++) != s.charAt(right--)) return false;
        }
        return true;
    }
}
```

Example:

Input: "hello world"
Output: "world hello"

Input: "madam"
Output: true

⌚ **Time Complexity: $O(N)$**
📦 **Space Complexity: $O(N)$**

PROBLEM 3

Problem Statement:

Given a numeric string, find the **largest odd number** that can be formed as a prefix.

Java Code:

```
import java.util.Scanner;

public class LargestOddNumber {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number string: ");
        String s = scanner.next();
        scanner.close();

        System.out.println("Largest Odd Number: " + largestOddNumber(s));
    }

    public static String largestOddNumber(String s) {
        for (int i = s.length() - 1; i >= 0; i--) {
            if ((s.charAt(i) - '0') % 2 == 1) return s.substring(0, i + 1);
        }
        return "";
    }
}
```

Example:

Input: "35427"
Output: "35427"

Input: "4206"
Output: ""

⌚ **Time Complexity: $O(N)$**
📦 **Space Complexity: $O(1)$**

PROBLEM 4:

Problem Statement:

Given an array of strings, find the **longest common prefix** among them.

Java Code:

```
import java.util.Scanner;

public class LongestCommonPrefix {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter number of words: ");
        int n = scanner.nextInt();
        String[] words = new String[n];
        System.out.println("Enter words:");
        for (int i = 0; i < n; i++) {
            words[i] = scanner.next();
        }
        scanner.close();

        System.out.println("Longest Common Prefix: " +
            longestCommonPrefix(words));
    }

    public static String longestCommonPrefix(String[] words) {
        if (words == null || words.length == 0) return "";
        String prefix = words[0];

        for (int i = 1; i < words.length; i++) {
            while (!words[i].startsWith(prefix)) {
                prefix = prefix.substring(0, prefix.length() - 1);
                if (prefix.isEmpty()) return "";
            }
        }
        return prefix;
    }
}
```

Example:

Input: ["flower", "flow", "flight"]
Output: "fl"

Input: ["dog", "racecar", "car"]
Output: ""

 **Time Complexity: $O(N * M)$**

 **Space Complexity: $O(1)$**

PROBLEM 5

Problem Statement:

Check if two strings are anagrams (contain the same characters with the same frequency).

Java Code:

```
import java.util.Arrays;
import java.util.Scanner;

public class AnagramCheck {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter first string: ");
        String s1 = scanner.next();
        System.out.print("Enter second string: ");
        String s2 = scanner.next();
        scanner.close();

        System.out.println("Are Anagrams: " + areAnagrams(s1, s2));
    }

    public static boolean areAnagrams(String s1, String s2) {
        if (s1.length() != s2.length()) return false;
        char[] arr1 = s1.toCharArray();
        char[] arr2 = s2.toCharArray();
        Arrays.sort(arr1);
        Arrays.sort(arr2);
        return Arrays.equals(arr1, arr2);
    }
}
```

Example:

Input: "listen", "silent"
Output: true

Input: "hello", "world"
Output: false

 **Time Complexity: $O(N \log N)$**

 **Space Complexity: $O(1)$**

Here are the solutions in your requested format:

PROBLEM 6

Problem Statement:

Write a Java program to sort characters in a string based on their frequency in descending order.

Java Code:

```
import java.util.*;

public class FrequencySort {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String s = scanner.nextLine();
        scanner.close();

        System.out.println("Sorted by frequency: " + frequencySort(s));
    }

    public static String frequencySort(String s) {
        Map<Character, Integer> freqMap = new HashMap<>();
        for (char c : s.toCharArray()) {
            freqMap.put(c, freqMap.getOrDefault(c, 0) + 1);
        }

        PriorityQueue<Character> maxHeap = new PriorityQueue<>((a, b) ->
            freqMap.get(b) - freqMap.get(a));
        maxHeap.addAll(freqMap.keySet());

        StringBuilder result = new StringBuilder();
        while (!maxHeap.isEmpty()) {
            char c = maxHeap.poll();
            result.append(String.valueOf(c).repeat(freqMap.get(c)));
        }
        return result.toString();
    }
}
```

Explanation:

1. The program takes a string input from the user.
 2. It counts the frequency of each character using a HashMap.
 3. A max heap (priority queue) is used to sort characters based on their frequency.
 4. Characters are added to the result string in order of highest frequency first.
-

PROBLEM 7

Problem Statement:

Write a Java program to find the maximum depth of nested parentheses in a given string.

Java Code:

```
import java.util.Scanner;

public class MaxDepthParentheses {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a string with parentheses: ");
    }
}
```

```

        String s = scanner.nextLine();
        scanner.close();

        System.out.println("Maximum Nesting Depth: " + maxDepth(s));
    }

    public static int maxDepth(String s) {
        int maxDepth = 0, currentDepth = 0;
        for (char c : s.toCharArray()) {
            if (c == '(') {
                currentDepth++;
                maxDepth = Math.max(maxDepth, currentDepth);
            } else if (c == ')') {
                currentDepth--;
            }
        }
        return maxDepth;
    }
}

```

Explanation:

1. The program takes a string as input.
 2. It iterates through the string, keeping track of the depth of open parentheses.
 3. The maximum depth encountered is recorded and returned.
-

PROBLEM 8

Problem Statement:

Write a Java program to convert a Roman numeral to an integer and vice versa.

Java Code:

```

java
CopyEdit
import java.util.*;

public class RomanConverter {
    private static final Map<Character, Integer> romanToIntMap = Map.of(
        'I', 1, 'V', 5, 'X', 10, 'L', 50, 'C', 100, 'D', 500, 'M', 1000
    );

    public static int romanToInt(String s) {
        int total = 0, prev = 0;
        for (int i = s.length() - 1; i >= 0; i--) {
            int curr = romanToIntMap.get(s.charAt(i));
            if (curr < prev) total -= curr;
            else total += curr;
            prev = curr;
        }
        return total;
    }
}

```



```

        public static String intToRoman(int num) {
            String[] romans = {"M", "CM", "D", "CD", "C", "XC", "L", "XL", "X",
"IX", "V", "IV", "I"};
            int[] values = {1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4,
1};
            StringBuilder result = new StringBuilder();

            for (int i = 0; i < values.length; i++) {
                while (num >= values[i]) {
                    result.append(romans[i]);
                    num -= values[i];
                }
            }
            return result.toString();
        }

        public static void main(String[] args) {
            Scanner scanner = new Scanner(System.in);
            System.out.print("Enter a Roman numeral: ");
            String roman = scanner.next();
            System.out.println("Integer value: " + romanToInt(roman));

            System.out.print("Enter an integer: ");
            int number = scanner.nextInt();
            System.out.println("Roman numeral: " + intToRoman(number));

            scanner.close();
        }
    }
}

```

Explanation:

1. The `romanToInt` method processes the string from right to left, adding or subtracting values based on precedence.
 2. The `intToRoman` method uses a mapping of values to Roman numerals and constructs the result.
 3. The program allows conversion both ways.
-

PROBLEM 9

Problem Statement:

Write a Java program to convert a string representation of an integer into an actual integer (Atoi function).

Java Code:

```

java
CopyEdit
import java.util.Scanner;

public class AtoiImplementation {
    public static int myAtoi(String s) {

```

```

s = s.trim();
if (s.isEmpty()) return 0;

int sign = 1, index = 0, result = 0;
if (s.charAt(0) == '-' || s.charAt(0) == '+') {
    sign = (s.charAt(0) == '-') ? -1 : 1;
    index++;
}

while (index < s.length() && Character.isDigit(s.charAt(index))) {
    int digit = s.charAt(index) - '0';
    if (result > (Integer.MAX_VALUE - digit) / 10) {
        return (sign == 1) ? Integer.MAX_VALUE : Integer.MIN_VALUE;
    }
    result = result * 10 + digit;
    index++;
}
return result * sign;
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Enter a string: ");
    String input = scanner.nextLine();
    System.out.println("Converted Integer: " + myAtoi(input));
    scanner.close();
}
}

```

Explanation:

1. The program trims the string to remove whitespace.
 2. It checks for a leading sign (+ or -).
 3. It processes digit characters while checking for overflow.
 4. The final integer is returned with the appropriate sign.
-

PROBLEM: 10

Explanation:

1. Uses **binary search** to locate a peak element efficiently.
2. If `arr[mid] > arr[mid+1]`, move left. Otherwise, move right.
3. Time Complexity: **O(log N)**.

Problem Statement:

Write a Java program to search for a target in a rotated sorted array that may contain duplicate values.

Java Code:

```
java
CopyEdit
import java.util.Scanner;

public class RotatedArraySearchII {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = scanner.nextInt();
        int[] arr = new int[n];

        System.out.println("Enter rotated sorted array elements (may
contain duplicates): ");
        for (int i = 0; i < n; i++) {
            arr[i] = scanner.nextInt();
        }

        System.out.print("Enter the number to search: ");
        int target = scanner.nextInt();
        scanner.close();

        boolean result = search(arr, target);
        System.out.println("Is " + target + " present? " + result);
    }

    public static boolean search(int[] arr, int target) {
        int left = 0, right = arr.length - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (arr[mid] == target) return true;

            if (arr[left] == arr[mid] && arr[mid] == arr[right]) {
                left++;
                right--;
            } else if (arr[left] <= arr[mid]) {
                if (arr[left] <= target && target < arr[mid]) right = mid -
1;
                else left = mid + 1;
            } else {
                if (arr[mid] < target && target <= arr[right]) left = mid +
1;
                else right = mid - 1;
            }
        }

        return false;
    }
}
```

Explanation:

1. Similar to **Search in Rotated Sorted Array I**, but handles **duplicates**.
2. If `arr[left] == arr[mid] == arr[right]`, increment `left` and decrement `right` to skip duplicates.
3. Uses **binary search** for **$O(\log N)$ worst-case $O(N)$ complexity**.

PROBLEM: 11

Problem Statement:

Write a Java program to find the minimum element in a rotated sorted array.

Java Code:

```
java
CopyEdit
import java.util.Scanner;

public class MinInRotatedArray {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = scanner.nextInt();
        int[] arr = new int[n];

        System.out.println("Enter rotated sorted array elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = scanner.nextInt();
        }
        scanner.close();

        int result = findMin(arr);
        System.out.println("Minimum element in rotated array: " + result);
    }

    public static int findMin(int[] arr) {
        int left = 0, right = arr.length - 1;

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (arr[mid] > arr[right]) left = mid + 1;
            else right = mid;
        }

        return arr[left];
    }
}
```

Explanation:

1. Uses **binary search** to find the **pivot** point.
 2. If $arr[mid] > arr[right]$, then **min is on the right half**.
 3. Else, **min is on the left half**.
 4. Runs in **$O(\log N)$ time**.
-

PROBLEM: 12

Problem Statement:

Write a Java program to determine how many times a sorted array has been rotated.

Java Code:

```
java
CopyEdit
import java.util.Scanner;

public class RotationCount {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = scanner.nextInt();
        int[] arr = new int[n];

        System.out.println("Enter rotated sorted array elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = scanner.nextInt();
        }
        scanner.close();

        int result = rotationCount(arr);
        System.out.println("Array is rotated " + result + " times.");
    }

    public static int rotationCount(int[] arr) {
        int left = 0, right = arr.length - 1;

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (arr[mid] > arr[right]) left = mid + 1;
            else right = mid;
        }

        return left;
    }
}
```

Explanation:

1. Finds **minimum element's index**, which is the **number of rotations**.
 2. Works the same way as **Find Minimum in Rotated Array**.
 3. Time Complexity: **$O(\log N)$** .
-

PROBLEM: 13

Problem Statement:

Write a Java program to find the **single non-repeating** element in a sorted array where every other element appears twice.

Java Code:

```
java
CopyEdit
import java.util.Scanner;

public class SingleElement {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = scanner.nextInt();
        int[] arr = new int[n];

        System.out.println("Enter sorted array elements (every number
appears twice except one): ");
        for (int i = 0; i < n; i++) {
            arr[i] = scanner.nextInt();
        }
        scanner.close();

        int result = findSingleElement(arr);
        System.out.println("Single non-repeating element: " + result);
    }

    public static int findSingleElement(int[] arr) {
        int left = 0, right = arr.length - 1;

        while (left < right) {
            int mid = left + (right - left) / 2;
            if (mid % 2 == 1) mid--;
            if (arr[mid] == arr[mid + 1]) left = mid + 2;
            else right = mid;
        }

        return arr[left];
    }
}
```

Explanation:

1. Uses **binary search** with **index-based checking**.
2. If `arr[mid] == arr[mid + 1]`, move right. Otherwise, move left.
3. Time Complexity: **$O(\log N)$** .

PROBLEM: 14

Problem Statement:

Write a Java program to find a **peak element** in an array. A peak element is an element that is strictly greater than its neighbors.

Java Code:

```
java
CopyEdit
import java.util.Scanner;
```

```

public class PeakElement {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = scanner.nextInt();
        int[] arr = new int[n];

        System.out.println("Enter array elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = scanner.nextInt();
        }
        scanner.close();

        int peakIndex = findPeakElement(arr);
        System.out.println("Peak element is at index: " + peakIndex + "
(Value: " + arr[peakIndex] + ")");
    }

    public static int findPeakElement(int[] arr) {
        int left = 0, right = arr.length - 1;

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (arr[mid] > arr[mid + 1]) right = mid;
            else left = mid + 1;
        }

        return left;
    }
}

```

Explanation:

1. Uses **binary search** to find a peak element.
2. If `arr[mid] > arr[mid + 1]`, the **peak lies in the left half**.
3. Otherwise, the **peak lies in the right half**.
4. Time Complexity: **$O(\log N)$** .

PROBLEM: 15

Problem Statement:

Write a Java program to find the **integer square root** of a given number using **binary search** in **$O(\log N)$** time complexity.

Java Code:

```

java
CopyEdit
import java.util.Scanner;

public class SquareRootBinarySearch {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number: ");
    }
}

```

```

        int num = scanner.nextInt();
        scanner.close();

        int sqrt = findSquareRoot(num);
        System.out.println("Square root of " + num + " is: " + sqrt);
    }

    public static int findSquareRoot(int num) {
        if (num == 0 || num == 1) return num;

        int left = 1, right = num, ans = 0;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (mid * mid == num) return mid;
            if (mid * mid < num) {
                ans = mid;
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return ans;
    }
}

```

Explanation:

1. We use **binary search** between 1 and num.
2. If $mid * mid == num$, return mid (perfect square).
3. If $mid * mid < num$, move **right** ($left = mid + 1$), store ans.
4. If $mid * mid > num$, move **left** ($right = mid - 1$).
5. Returns **integer square root** (floor value).

Example:

vbnet

CopyEdit

Input: num = 17

Output: Square root of 17 is: 4

 **Time Complexity: $O(\log N)$.** Efficient for large numbers!

PROBLEM: 16

Problem Statement:

Write a Java program to find the **N-th root** of a number using **binary search**.

Java Code:

java

CopyEdit


```

import java.util.Scanner;

public class NthRootBinarySearch {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter the number: ");
        int num = scanner.nextInt();
        System.out.print("Enter the value of N: ");
        int n = scanner.nextInt();
        scanner.close();

        double nthRoot = findNthRoot(num, n);
        System.out.println("The " + n + "th root of " + num + " is: " +
nthRoot);
    }

    public static double findNthRoot(int num, int n) {
        double low = 1, high = num, epsilon = 1e-6;
        while ((high - low) > epsilon) {
            double mid = (low + high) / 2.0;
            if (Math.pow(mid, n) < num) low = mid;
            else high = mid;
        }
        return high;
    }
}

```

Explanation:

1. Uses **binary search** in **floating point precision**.
2. If $\text{mid}^n < \text{num}$, move $\text{low} = \text{mid}$.
3. If $\text{mid}^n > \text{num}$, move $\text{high} = \text{mid}$.
4. Returns **nth root with precision** ($\text{epsilon} = 10^{-6}$).

Example:

vbnet

CopyEdit

Input: num = 16, n = 4

Output: The 4th root of 16 is: 2.000000

 **Time Complexity: $O(\log N)$ using floating point binary search.**

PROBLEM: 17

Problem Statement:

Given piles of bananas and h hours, find the **minimum eating speed** (k) such that Koko eats all bananas within h hours.

Java Code:

```
java
CopyEdit
import java.util.Scanner;

public class KokoEatingBananas {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter number of piles: ");
        int n = scanner.nextInt();
        int[] piles = new int[n];

        System.out.println("Enter banana piles: ");
        for (int i = 0; i < n; i++) {
            piles[i] = scanner.nextInt();
        }

        System.out.print("Enter total hours (h): ");
        int h = scanner.nextInt();
        scanner.close();

        int minSpeed = minEatingSpeed(piles, h);
        System.out.println("Minimum eating speed: " + minSpeed);
    }

    public static int minEatingSpeed(int[] piles, int h) {
        int left = 1, right = Integer.MAX_VALUE;
        while (left < right) {
            int mid = left + (right - left) / 2;
            if (canEatAll(piles, mid, h)) right = mid;
            else left = mid + 1;
        }
        return left;
    }

    private static boolean canEatAll(int[] piles, int k, int h) {
        int hours = 0;
        for (int pile : piles) {
            hours += (pile + k - 1) / k;
        }
        return hours <= h;
    }
}
```

Explanation:

1. Uses **binary search** for k (eating speed).
2. Check if Koko can **finish in h hours**.
3. If `canEatAll(piles, k, h)`, decrease k .
4. If not, increase k .

Example:

```
makefile
CopyEdit
Input:  piles = [3,6,7,11], h = 8
Output: Minimum eating speed: 4
```

 **Time Complexity: $O(N \log M)$** (M is max pile size).

PROBLEM: 18

Problem Statement:

Find the **minimum number of days** to make m bouquets, each requiring k adjacent flowers to bloom.

Java Code:

```
java
CopyEdit
import java.util.Scanner;
import java.util.Arrays;

public class MinDaysBouquets {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter number of flowers: ");
        int n = scanner.nextInt();
        int[] bloomDay = new int[n];

        System.out.println("Enter bloom days: ");
        for (int i = 0; i < n; i++) {
            bloomDay[i] = scanner.nextInt();
        }

        System.out.print("Enter bouquets needed: ");
        int m = scanner.nextInt();
        System.out.print("Enter flowers per bouquet: ");
        int k = scanner.nextInt();
        scanner.close();

        int minDays = minDaysToMakeBouquets(bloomDay, m, k);
        System.out.println("Minimum days to make " + m + " bouquets: " +
minDays);
    }

    public static int minDaysToMakeBouquets(int[] bloomDay, int m, int k) {
        if (bloomDay.length < m * k) return -1;

        int left = Arrays.stream(bloomDay).min().getAsInt();
        int right = Arrays.stream(bloomDay).max().getAsInt();

        while (left < right) {
            int mid = left + (right - left) / 2;
            if (canMakeBouquets(bloomDay, mid, m, k)) right = mid;
            else left = mid + 1;
        }
        return left;
    }
}
```

```

        private static boolean canMakeBouquets(int[] bloomDay, int days, int m,
int k) {
    int flowers = 0, bouquets = 0;
    for (int bloom : bloomDay) {
        if (bloom <= days) flowers++;
        else flowers = 0;

        if (flowers == k) {
            bouquets++;
            flowers = 0;
        }
    }
    return bouquets >= m;
}
}

```

Explanation:

1. Binary search for `days`.
2. Check if `days` can make `m` bouquets.
3. Adjust search range accordingly.

PROBLEM: 19

Problem Statement:

Given an array `nums` and a threshold `t`, find the **smallest divisor** such that the sum of all elements divided by it does not exceed `t`.

Java Code:

```

java
CopyEdit
import java.util.Scanner;

public class SmallestDivisor {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter array size: ");
        int n = scanner.nextInt();
        int[] nums = new int[n];

        System.out.println("Enter array elements: ");
        for (int i = 0; i < n; i++) {
            nums[i] = scanner.nextInt();
        }

        System.out.print("Enter threshold: ");
        int threshold = scanner.nextInt();
        scanner.close();

        int result = findSmallestDivisor(nums, threshold);
        System.out.println("Smallest Divisor: " + result);
    }

    public static int findSmallestDivisor(int[] nums, int threshold) {
        int left = 1, right = Integer.MAX_VALUE;
        while (left < right) {

```

```

        int mid = left + (right - left) / 2;
        if (sumDividedBy(nums, mid) <= threshold) right = mid;
        else left = mid + 1;
    }
    return left;
}

private static int sumDividedBy(int[] nums, int divisor) {
    int sum = 0;
    for (int num : nums) {
        sum += (num + divisor - 1) / divisor;
    }
    return sum;
}
}

```

Explanation:

1. **Binary search** on possible divisors (1 to max element).
2. If $\text{sum} \leq \text{threshold}$, decrease divisor ($\text{right} = \text{mid}$).
3. Otherwise, increase divisor ($\text{left} = \text{mid} + 1$).

Example:

```

makefile
CopyEdit
Input: nums = [1,2,5,9], threshold = 6
Output: Smallest Divisor: 5

```

 **Time Complexity:** $O(N \log M)$ (M is max element).

PROBLEM: 20

Problem Statement:

Given an array representing **weights of packages**, find the **minimum ship capacity** to ship them within D days.

Java Code:

```

java
CopyEdit
import java.util.Scanner;

public class ShipCapacity {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter number of packages: ");
        int n = scanner.nextInt();
        int[] weights = new int[n];

        System.out.println("Enter package weights: ");
    }
}

```

```

        for (int i = 0; i < n; i++) {
            weights[i] = scanner.nextInt();
        }

        System.out.print("Enter number of days: ");
        int D = scanner.nextInt();
        scanner.close();

        int result = minShipCapacity(weights, D);
        System.out.println("Minimum Ship Capacity: " + result);
    }

    public static int minShipCapacity(int[] weights, int D) {
        int left = 0, right = 0;
        for (int weight : weights) {
            left = Math.max(left, weight);
            right += weight;
        }

        while (left < right) {
            int mid = left + (right - left) / 2;
            if (canShip(weights, D, mid)) right = mid;
            else left = mid + 1;
        }
        return left;
    }

    private static boolean canShip(int[] weights, int D, int capacity) {
        int days = 1, load = 0;
        for (int weight : weights) {
            if (load + weight > capacity) {
                days++;
                load = 0;
            }
            load += weight;
        }
        return days <= D;
    }
}

```

Explanation:

1. **Binary search** between $\max(\text{weight})$ (min capacity) and $\text{sum}(\text{weights})$ (max capacity).
2. Check if a **capacity** can ship in D days.
3. If `canShip()`, decrease capacity ($\text{right} = \text{mid}$).
4. Else, increase capacity ($\text{left} = \text{mid} + 1$).

Example:

```

makefile
CopyEdit
Input: weights = [1,2,3,4,5,6,7,8,9,10], D = 5
Output: Minimum Ship Capacity: 15

```

 **Time Complexity:** $O(N \log M)$ (M is sum of weights).

PROBLEM: 21

Problem Statement:

Find the **K-th missing positive number** from a sorted array of unique positive integers.

Java Code:

```
java
CopyEdit
import java.util.Scanner;

public class KthMissingPositive {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter array size: ");
        int n = scanner.nextInt();
        int[] arr = new int[n];

        System.out.println("Enter sorted array elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = scanner.nextInt();
        }

        System.out.print("Enter K: ");
        int k = scanner.nextInt();
        scanner.close();

        int result = findKthMissing(arr, k);
        System.out.println("Kth Missing Positive Number: " + result);
    }

    public static int findKthMissing(int[] arr, int k) {
        int left = 0, right = arr.length - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            int missing = arr[mid] - (mid + 1);
            if (missing < k) left = mid + 1;
            else right = mid - 1;
        }
        return left + k;
    }
}
```

Explanation:

1. **Binary search** on $arr[i] - (i + 1)$.
2. If missing numbers so far $< k$, increase left.
3. Otherwise, decrease right.
4. Final result is $left + k$.

Example:

mathematica

CopyEdit

Input: arr = [2,3,4,7,11], k = 5

Output: Kth Missing Positive Number: 9

 **Time Complexity: $O(\log N)$.**

PROBLEM: 22

Problem Statement:

Given n stalls and c cows, place the cows in the stalls so that the **minimum distance** between any two cows is **maximized**.

Java Code:

```
java
CopyEdit
import java.util.Arrays;
import java.util.Scanner;

public class AggressiveCows {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter number of stalls: ");
        int n = scanner.nextInt();
        int[] stalls = new int[n];

        System.out.println("Enter stall positions: ");
        for (int i = 0; i < n; i++) {
            stalls[i] = scanner.nextInt();
        }

        System.out.print("Enter number of cows: ");
        int cows = scanner.nextInt();
        scanner.close();

        int result = findMaxMinDistance(stalls, cows);
        System.out.println("Largest minimum distance: " + result);
    }

    public static int findMaxMinDistance(int[] stalls, int cows) {
        Arrays.sort(stalls);
        int left = 1, right = stalls[n - 1] - stalls[0];
        int result = 0;

        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (canPlaceCows(stalls, cows, mid)) {
                result = mid;
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return result;
    }
}
```



```

private static boolean canPlaceCows(int[] stalls, int cows, int minDist)
{
    int count = 1, lastPlaced = stalls[0];
    for (int i = 1; i < stalls.length; i++) {
        if (stalls[i] - lastPlaced >= minDist) {
            count++;
            lastPlaced = stalls[i];
        }
        if (count == cows) return true;
    }
    return false;
}
}

```

Explanation:

1. **Sort the stalls** in ascending order.
 2. **Binary search** on possible distances.
 3. Use `canPlaceCows()` to check if cows can be placed with `minDist`.
 4. If yes, try a larger distance (`left = mid + 1`); otherwise, reduce (`right = mid - 1`).
-

Example:

```

makefile
CopyEdit
Input: stalls = [1, 2, 8, 4, 9], cows = 3
Output: Largest minimum distance: 3

```

 **Time Complexity: $O(N \log M)$** (M is max stall position).

PROBLEM: 23

Problem Statement:

Allocate n books to m students such that the **maximum pages assigned to a student** is minimized.

Java Code:

```

java
CopyEdit
import java.util.Scanner;

public class BookAllocation {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter number of books: ");
        int n = scanner.nextInt();
        int[] books = new int[n];

        System.out.println("Enter pages in books: ");
        for (int i = 0; i < n; i++) {
            books[i] = scanner.nextInt();
        }
    }
}

```

```

    }

    System.out.print("Enter number of students: ");
    int students = scanner.nextInt();
    scanner.close();

    int result = findMinimumPages(books, students);
    System.out.println("Minimum max pages allocated: " + result);
}

public static int findMinimumPages(int[] books, int students) {
    int left = 0, right = 0;
    for (int pages : books) {
        left = Math.max(left, pages);
        right += pages;
    }

    while (left < right) {
        int mid = left + (right - left) / 2;
        if (canAllocate(books, students, mid)) right = mid;
        else left = mid + 1;
    }
    return left;
}

private static boolean canAllocate(int[] books, int students, int
maxPages) {
    int studentCount = 1, pagesSum = 0;
    for (int pages : books) {
        if (pagesSum + pages > maxPages) {
            studentCount++;
            pagesSum = 0;
        }
        pagesSum += pages;
    }
    return studentCount <= students;
}
}

```

Explanation:

1. **Binary search** between `max(books)` and `sum(books)`.
2. Check if `mid` pages can be allocated using `canAllocate()`.
3. If `canAllocate()`, try a smaller value (`right = mid`).
4. Else, increase pages (`left = mid + 1`).

Example:

```

makefile
CopyEdit
Input: books = [10, 20, 30, 40], students = 2
Output: Minimum max pages allocated: 60

```

 **Time Complexity: $O(N \log M)$.**

PROBLEM: 24

Problem Statement:

Given two **sorted arrays**, find their **combined median** efficiently.

Java Code:

```
java
CopyEdit
public class MedianTwoSortedArrays {
    public static void main(String[] args) {
        int[] nums1 = {1, 3};
        int[] nums2 = {2};
        System.out.println("Median: " + findMedianSortedArrays(nums1,
nums2));
    }

    public static double findMedianSortedArrays(int[] nums1, int[] nums2) {
        if (nums1.length > nums2.length) return
findMedianSortedArrays(nums2, nums1);

        int x = nums1.length, y = nums2.length;
        int low = 0, high = x;

        while (low <= high) {
            int partitionX = (low + high) / 2;
            int partitionY = (x + y + 1) / 2 - partitionX;

            int maxX = (partitionX == 0) ? Integer.MIN_VALUE :
nums1[partitionX - 1];
            int minX = (partitionX == x) ? Integer.MAX_VALUE :
nums1[partitionX];

            int maxY = (partitionY == 0) ? Integer.MIN_VALUE :
nums2[partitionY - 1];
            int minY = (partitionY == y) ? Integer.MAX_VALUE :
nums2[partitionY];

            if (maxX <= minY && maxY <= minX) {
                if ((x + y) % 2 == 0) return ((double)Math.max(maxX, maxY)
+ Math.min(minX, minY)) / 2;
                else return (double)Math.max(maxX, maxY);
            } else if (maxX > minY) high = partitionX - 1;
            else low = partitionX + 1;
        }
        throw new IllegalArgumentException();
    }
}
```

Explanation:

1. **Binary search** on the smaller array for an optimal partition.
2. **Partitions** divide both arrays into **left** (smaller) and **right** (larger).
3. Ensure $\max(\text{left}) \leq \min(\text{right})$.
4. If **even length**, return **average** of middle elements.
5. If **odd length**, return the **larger middle element**.

Example:

makefile

CopyEdit

Input: nums1 = [1, 3], nums2 = [2]

Output: Median: 2.0

 **Time Complexity: $O(\log(\min(N, M)))$.**

PROBLEM: 25

Problem Statement:

Given n stalls and c cows, place the cows in the stalls so that the **minimum distance** between any two cows is **maximized**.

Java Code:

java

CopyEdit

```
import java.util.Arrays;
```

```
import java.util.Scanner;
```

```
public class AggressiveCows {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter number of stalls: ");
        int n = scanner.nextInt();
        int[] stalls = new int[n];

        System.out.println("Enter stall positions: ");
        for (int i = 0; i < n; i++) {
            stalls[i] = scanner.nextInt();
        }

        System.out.print("Enter number of cows: ");
        int cows = scanner.nextInt();
        scanner.close();

        int result = findMaxMinDistance(stalls, cows);
        System.out.println("Largest minimum distance: " + result);
    }

    public static int findMaxMinDistance(int[] stalls, int cows) {
        Arrays.sort(stalls);
        int left = 1, right = stalls[n - 1] - stalls[0];
        int result = 0;

        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (canPlaceCows(stalls, cows, mid)) {
                result = mid;
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
    }
}
```

```

        return result;
    }

    private static boolean canPlaceCows(int[] stalls, int cows, int minDist)
    {
        int count = 1, lastPlaced = stalls[0];
        for (int i = 1; i < stalls.length; i++) {
            if (stalls[i] - lastPlaced >= minDist) {
                count++;
                lastPlaced = stalls[i];
            }
            if (count == cows) return true;
        }
        return false;
    }
}

```

Explanation:

5. **Sort the stalls** in ascending order.
 6. **Binary search** on possible distances.
 7. Use `canPlaceCows()` to check if cows can be placed with `minDist`.
 8. If yes, try a larger distance (`left = mid + 1`); otherwise, reduce (`right = mid - 1`).
-

Example:

```

makefile
CopyEdit
Input: stalls = [1, 2, 8, 4, 9], cows = 3
Output: Largest minimum distance: 3

```

 **Time Complexity: $O(N \log M)$** (M is max stall position).

PROBLEM: 26

Problem Statement:

Allocate n books to m students such that the **maximum pages assigned to a student** is minimized.

Java Code:

```

java
CopyEdit
import java.util.Scanner;

public class BookAllocation {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter number of books: ");
        int n = scanner.nextInt();
        int[] books = new int[n];
    }
}

```

```

        System.out.println("Enter pages in books: ");
        for (int i = 0; i < n; i++) {
            books[i] = scanner.nextInt();
        }

        System.out.print("Enter number of students: ");
        int students = scanner.nextInt();
        scanner.close();

        int result = findMinimumPages(books, students);
        System.out.println("Minimum max pages allocated: " + result);
    }

    public static int findMinimumPages(int[] books, int students) {
        int left = 0, right = 0;
        for (int pages : books) {
            left = Math.max(left, pages);
            right += pages;
        }

        while (left < right) {
            int mid = left + (right - left) / 2;
            if (canAllocate(books, students, mid)) right = mid;
            else left = mid + 1;
        }
        return left;
    }

    private static boolean canAllocate(int[] books, int students, int
maxPages) {
        int studentCount = 1, pagesSum = 0;
        for (int pages : books) {
            if (pagesSum + pages > maxPages) {
                studentCount++;
                pagesSum = 0;
            }
            pagesSum += pages;
        }
        return studentCount <= students;
    }
}

```

Explanation:

5. **Binary search** between `max(books)` and `sum(books)`.
6. Check if `mid` pages can be allocated using `canAllocate()`.
7. If `canAllocate()`, try a smaller value (`right = mid`).
8. Else, increase pages (`left = mid + 1`).

Example:

```

makefile
CopyEdit
Input: books = [10, 20, 30, 40], students = 2
Output: Minimum max pages allocated: 60

```

 **Time Complexity: $O(N \log M)$.**

PROBLEM: 27

Problem Statement:

Given two **sorted arrays**, find their **combined median** efficiently.

Java Code:

```
java
CopyEdit
public class MedianTwoSortedArrays {
    public static void main(String[] args) {
        int[] nums1 = {1, 3};
        int[] nums2 = {2};
        System.out.println("Median: " + findMedianSortedArrays(nums1,
nums2));
    }

    public static double findMedianSortedArrays(int[] nums1, int[] nums2) {
        if (nums1.length > nums2.length) return
findMedianSortedArrays(nums2, nums1);

        int x = nums1.length, y = nums2.length;
        int low = 0, high = x;

        while (low <= high) {
            int partitionX = (low + high) / 2;
            int partitionY = (x + y + 1) / 2 - partitionX;

            int maxX = (partitionX == 0) ? Integer.MIN_VALUE :
nums1[partitionX - 1];
            int minX = (partitionX == x) ? Integer.MAX_VALUE :
nums1[partitionX];

            int maxY = (partitionY == 0) ? Integer.MIN_VALUE :
nums2[partitionY - 1];
            int minY = (partitionY == y) ? Integer.MAX_VALUE :
nums2[partitionY];

            if (maxX <= minY && maxY <= minX) {
                if ((x + y) % 2 == 0) return ((double)Math.max(maxX, maxY)
+ Math.min(minX, minY)) / 2;
                else return (double)Math.max(maxX, maxY);
            } else if (maxX > minY) high = partitionX - 1;
            else low = partitionX + 1;
        }
        throw new IllegalArgumentException();
    }
}
```

Explanation:

6. **Binary search** on the smaller array for an optimal partition.

7. **Partitions** divide both arrays into **left** (smaller) and **right** (larger).
8. Ensure $\max(\text{left}) \leq \min(\text{right})$.
9. If **even length**, return **average** of middle elements.
10. If **odd length**, return the **larger middle element**.

Example:

```
makefile
CopyEdit
Input: nums1 = [1, 3], nums2 = [2]
Output: Median: 2.0
```

 **Time Complexity: $O(\log(\min(N, M)))$.**

PROBLEM: 28

Problem Statement:

Given an array `nums` and an integer `m`, partition the array into `m` non-empty subarrays such that the **largest sum among these subarrays is minimized**.

Java Code:

```
java
CopyEdit
import java.util.Scanner;

public class SplitArrayLargestSum {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter number of elements: ");
        int n = scanner.nextInt();
        int[] nums = new int[n];

        System.out.println("Enter array elements: ");
        for (int i = 0; i < n; i++) {
            nums[i] = scanner.nextInt();
        }

        System.out.print("Enter number of subarrays: ");
        int m = scanner.nextInt();
        scanner.close();

        int result = splitArray(nums, m);
        System.out.println("Minimum largest sum: " + result);
    }

    public static int splitArray(int[] nums, int m) {
        int left = 0, right = 0;
        for (int num : nums) {
            left = Math.max(left, num);
            right += num;
        }

        while (left < right) {
            int mid = left + (right - left) / 2;
```



```

        if (canSplit(nums, m, mid)) right = mid;
        else left = mid + 1;
    }
    return left;
}

private static boolean canSplit(int[] nums, int m, int maxSum) {
    int count = 1, sum = 0;
    for (int num : nums) {
        if (sum + num > maxSum) {
            count++;
            sum = 0;
        }
        sum += num;
    }
    return count <= m;
}
}

```

Explanation:

1. **Binary search** for the minimum possible largest sum.
 2. **Check feasibility** using `canSplit()`.
 3. If `canSplit()` returns `true`, search for a smaller maximum sum.
-

Example:

```

makefile
CopyEdit
Input: nums = [7, 2, 5, 10, 8], m = 2
Output: Minimum largest sum: 18

```

 **Time Complexity: $O(N \log S)$** (S = sum of elements).

PROBLEM: 29

Problem Statement:

Given k painters and n boards, each with a given length, assign boards to painters such that the **maximum time** taken by any painter is minimized.

Java Code:

```

java
CopyEdit
import java.util.Scanner;

public class PaintersPartition {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter number of boards: ");
        int n = scanner.nextInt();
    }
}

```

```

int[] boards = new int[n];

System.out.println("Enter board lengths: ");
for (int i = 0; i < n; i++) {
    boards[i] = scanner.nextInt();
}

System.out.print("Enter number of painters: ");
int k = scanner.nextInt();
scanner.close();

int result = minTime(boards, k);
System.out.println("Minimum time to paint all boards: " + result);
}

public static int minTime(int[] boards, int k) {
    int left = 0, right = 0;
    for (int board : boards) {
        left = Math.max(left, board);
        right += board;
    }

    while (left < right) {
        int mid = left + (right - left) / 2;
        if (canPaint(boards, k, mid)) right = mid;
        else left = mid + 1;
    }
    return left;
}

private static boolean canPaint(int[] boards, int k, int maxTime) {
    int painters = 1, sum = 0;
    for (int board : boards) {
        if (sum + board > maxTime) {
            painters++;
            sum = 0;
        }
        sum += board;
    }
    return painters <= k;
}
}

```

Explanation:

1. **Binary search** for the minimum possible max time.
2. **Check feasibility** using `canPaint()`.
3. Adjust search space accordingly.

Example:

```

pgsql
CopyEdit
Input: boards = [10, 20, 30, 40], painters = 2
Output: Minimum time to paint all boards: 60

```

 **Time Complexity: $O(N \log S)$.**

PROBLEM: 30

Problem Statement:

Given n gas stations at positions `stations[]`, and K additional stations to be placed, **minimize the maximum distance** between adjacent stations.

Java Code:

```
java
CopyEdit
import java.util.Arrays;
import java.util.Scanner;

public class MinimizeMaxDistance {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter number of gas stations: ");
        int n = scanner.nextInt();
        double[] stations = new double[n];

        System.out.println("Enter gas station positions: ");
        for (int i = 0; i < n; i++) {
            stations[i] = scanner.nextDouble();
        }

        System.out.print("Enter number of new stations: ");
        int K = scanner.nextInt();
        scanner.close();

        double result = minMaxDistance(stations, K);
        System.out.println("Minimum possible max distance: " + result);
    }

    public static double minMaxDistance(double[] stations, int K) {
        Arrays.sort(stations);
        double left = 0, right = stations[stations.length - 1] - stations[0];

        while (right - left > 1e-6) {
            double mid = (left + right) / 2;
            if (canPlace(stations, K, mid)) right = mid;
            else left = mid;
        }
        return right;
    }

    private static boolean canPlace(double[] stations, int K, double maxDist) {
        int count = 0;
        for (int i = 1; i < stations.length; i++) {
            count += (int) ((stations[i] - stations[i - 1]) / maxDist);
        }
        return count <= K;
    }
}
```

```
}
```

Explanation:

1. **Binary search** on the possible max distance.
 2. **Check feasibility** using `canPlace()`.
 3. Adjust the search space.
-

Example:

makefile

CopyEdit

Input: stations = [1, 2, 3, 6, 7], K = 2

Output: Minimum possible max distance: 1.5

 **Time Complexity: $O(N \log D)$** (D = max distance).

PROBLEM: 31

Problem Statement:

Find the **Kth smallest element** in two sorted arrays A[] and B[].

Java Code:

java

CopyEdit

```
public class KthElementTwoSorted {
    public static void main(String[] args) {
        int[] A = {2, 3, 6, 7, 9};
        int[] B = {1, 4, 8, 10};
        int K = 5;

        System.out.println("Kth element: " + findKthElement(A, B, K));
    }

    public static int findKthElement(int[] A, int[] B, int K) {
        if (A.length > B.length) return findKthElement(B, A, K);

        int low = Math.max(0, K - B.length), high = Math.min(K, A.length);
        while (low < high) {
            int midA = (low + high) / 2;
            int midB = K - midA;

            if (A[midA] < B[midB - 1]) low = midA + 1;
            else high = midA;
        }
        return Math.max(low > 0 ? A[low - 1] : Integer.MIN_VALUE, K - low >
0 ? B[K - low - 1] : Integer.MIN_VALUE);
    }
}
```

Explanation:

1. **Binary search** on A[] to find the **Kth element**.
2. Adjust partitions accordingly.

Example:

makefile

CopyEdit

Input: A = [2, 3, 6, 7, 9], B = [1, 4, 8, 10], K = 5

Output: Kth element: 6

 **Time Complexity: $O(\log \min(N, M))$.**

PROBLEM: 32

Problem Statement:

Given a binary matrix `mat[][]` of size $N \times M$, where each row is sorted in **non-decreasing** order, find the **row index** with the **maximum number of 1's**. If multiple rows have the same maximum count, return the index of the first such row.

Java Code:

```
public class RowWithMaxOnes {
    public static void main(String[] args) {
        int[][] mat = {
            {0, 0, 1, 1},
            {0, 1, 1, 1},
            {0, 0, 0, 1},
            {1, 1, 1, 1}
        };

        System.out.println("Row with maximum 1s: " + rowWithMax1s(mat));
    }

    public static int rowWithMax1s(int[][] mat) {
        int maxRowIndex = -1, maxCount = 0;
        int cols = mat[0].length;

        for (int i = 0; i < mat.length; i++) {
            int onesCount = cols - firstOneIndex(mat[i]);
            if (onesCount > maxCount) {
                maxCount = onesCount;
                maxRowIndex = i;
            }
        }
        return maxRowIndex;
    }
}
```

```

private static int firstOneIndex(int[] row) {
    int left = 0, right = row.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (row[mid] == 1) right = mid - 1;
        else left = mid + 1;
    }
    return left;
}
}

```

Explanation:

1. **Use binary search** to find the first occurrence of 1 in each row.
 2. Compute the **number of 1's** in that row.
 3. **Track the row** with the highest count.
-

Example:

Input:

```

mat = [
    [0, 0, 1, 1],
    [0, 1, 1, 1],
    [0, 0, 0, 1],
    [1, 1, 1, 1]
]

```

Output: Row with maximum 1s: 3

 **Time Complexity: $O(N \log M)$.**

PROBLEM: 33

Problem Statement:

Given an $N \times M$ matrix where:

- Each row is sorted from **left to right**.
 - The first integer of each row is **greater than the last integer of the previous row**.
- Determine whether a target integer exists in the matrix.
-

Java Code:

```

public class Search2DMatrix {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 3, 5, 7},
            {10, 11, 16, 20},
            {23, 30, 34, 60}
        };
    }
}

```

```
        int target = 16;

        System.out.println("Target found: " + searchMatrix(matrix, target));
    }

    public static boolean searchMatrix(int[][] matrix, int target) {
        int rows = matrix.length, cols = matrix[0].length;
        int left = 0, right = rows * cols - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;
            int midValue = matrix[mid / cols][mid % cols];

            if (midValue == target) return true;
            else if (midValue < target) left = mid + 1;
            else right = mid - 1;
        }
        return false;
    }
}
```

Explanation:

1. **Flatten the matrix** into a **1D array** using indexing.
 2. **Apply binary search** to locate the target.
-

Example:

```
Input: matrix = [
    [1, 3, 5, 7],
    [10, 11, 16, 20],
    [23, 30, 34, 60]
], target = 16
Output: Target found: true
```

 **Time Complexity: $O(\log(N*M))$.**

PROBLEM: 34

Problem Statement:

Given an $N \times M$ matrix where:

- Each row is sorted from **left to right**.
 - Each column is sorted from **top to bottom**.
- Determine whether a target integer exists in the matrix.
-

Java Code:

```
public class SearchSortedMatrix {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 4, 7, 11},
            {2, 5, 8, 12},
            {3, 6, 9, 16},
            {10, 13, 14, 17}
        };
        int target = 5;

        System.out.println("Target found: " + searchMatrix(matrix, target));
    }

    public static boolean searchMatrix(int[][] matrix, int target) {
        int row = 0, col = matrix[0].length - 1;

        while (row < matrix.length && col >= 0) {
            if (matrix[row][col] == target) return true;
            else if (matrix[row][col] < target) row++;
            else col--;
        }
        return false;
    }
}
```

Explanation:

1. **Start from the top-right corner.**
 2. **Move left if the current value is greater than the target.**
 3. **Move down if the current value is smaller than the target.**
-

Example:

```
Input: matrix = [
    [1, 4, 7, 11],
    [2, 5, 8, 12],
    [3, 6, 9, 16],
    [10, 13, 14, 17]
], target = 5
Output: Target found: true
```

 **Time Complexity: $O(N + M)$.**

PROBLEM: 35

Problem Statement:

A **peak element** in a 2D matrix is an element that is **greater than all of its 4 neighbors**. Given an $N \times M$ matrix, find **any** peak element.

Java Code:

```
public class FindPeakElement2D {
    public static void main(String[] args) {
        int[][] matrix = {
            {10, 20, 15},
            {21, 30, 14},
            {7, 16, 32}
        };

        int[] peak = findPeakElement(matrix);
        System.out.println("Peak Element found at: (" + peak[0] + ", " +
peak[1] + ")");
    }

    public static int[] findPeakElement(int[][] matrix) {
        int rows = matrix.length, cols = matrix[0].length;
        int left = 0, right = cols - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;
            int rowMaxIndex = getMaxIndex(matrix, mid);

            int leftValue = (mid > 0) ? matrix[rowMaxIndex][mid - 1] :
Integer.MIN_VALUE;
            int rightValue = (mid < cols - 1) ? matrix[rowMaxIndex][mid +
1] : Integer.MIN_VALUE;

            if (matrix[rowMaxIndex][mid] > leftValue &&
matrix[rowMaxIndex][mid] > rightValue) {
                return new int[]{rowMaxIndex, mid};
            } else if (matrix[rowMaxIndex][mid] < leftValue) right = mid -
1;
            else left = mid + 1;
        }
        return new int[]{-1, -1};
    }

    private static int getMaxIndex(int[][] matrix, int col) {
        int maxIndex = 0;
        for (int i = 1; i < matrix.length; i++) {
            if (matrix[i][col] > matrix[maxIndex][col]) maxIndex = i;
        }
        return maxIndex;
    }
}
```

Explanation:

1. Use **binary search on columns**.
 2. Find the **row-wise max in the middle column**.
 3. **Adjust search space** accordingly.
-

 **Time Complexity: $O(N \log M)$.**

```

    }

    public static void moveZeros(int[] arr) {
        int nonZeroIndex = 0;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] != 0) {
                arr[nonZeroIndex++] = arr[i];
            }
        }
        while (nonZeroIndex < arr.length) {
            arr[nonZeroIndex++] = 0;
        }
    }
}

```

PROBLEM: 36

Problem Statement:

Given an $N \times M$ **row-wise sorted** matrix, find its **median**.

- The median is the middle element when all elements are sorted.
- If the total number of elements is odd, return the middle element.
- If the total number of elements is even, return the smaller of the two middle elements.

Java Code:

```

java
CopyEdit
import java.util.Arrays;

public class MatrixMedian {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 3, 5},
            {2, 6, 9},
            {3, 6, 9}
        };

        System.out.println("Median of matrix: " + findMedian(matrix));
    }

    public static int findMedian(int[][] matrix) {
        int rows = matrix.length, cols = matrix[0].length;
        int min = Integer.MAX_VALUE, max = Integer.MIN_VALUE;

        // Find the global min and max in the matrix
        for (int i = 0; i < rows; i++) {
            min = Math.min(min, matrix[i][0]); // First element of each row
            max = Math.max(max, matrix[i][cols - 1]); // Last element of
each row
        }

        int desiredCount = (rows * cols) / 2; // Middle index
        while (min < max) {

```

```

        int mid = min + (max - min) / 2;
        int count = countLessOrEqual(matrix, mid);

        if (count <= desiredCount) min = mid + 1;
        else max = mid;
    }
    return min;
}

private static int countLessOrEqual(int[][] matrix, int num) {
    int count = 0;
    for (int[] row : matrix) {
        count += countSmallerEqual(row, num);
    }
    return count;
}

private static int countSmallerEqual(int[] row, int num) {
    int left = 0, right = row.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (row[mid] <= num) left = mid + 1;
        else right = mid - 1;
    }
    return left; // Position of first greater element = count of ≤ num
elements
    }
}

```

Explanation:

1. **Find the min & max** values in the matrix.
 2. **Use binary search** on the range [min, max].
 3. **Count numbers ≤ mid** in the matrix using binary search.
 4. If count is ≤ **median index**, search the **right half**; otherwise, search the **left half**.
-

 **Time Complexity: $O(N \log M \log \text{max-min})$.**

 **Space Complexity: $O(1)$** (No extra space used).

Congratulations on completing the PDF created by SYNTAX ERROR, also known as Abhishek Rathor! Your dedication and perseverance are truly commendable. For more insights and updates, feel free to connect on Instagram at SYNTAX ERROR. Keep up the great work!